UAlg – ISE – DEE
Algoritmos e Estruturas de Dados
Ordenação de conjunto de Inteiros

Vasco Matos
UALg – Universidade do Algarve
Faro

a85234@ualg.pt

Resumo
No seguinte relatório encontra-se a descrição das tecnologias e funcionalidades de um programa ordenação em JAVA que recebe um ficheiro numérico em formato texto (.txt) ou binário e ordena exportando num novo ficheiro os valores ordenados.

1. INTRODUÇÃO
Para o desenvolvimento do projeto foi utilizado como plataforma o VSCode juntamente de um repositório Git. Java foi a linguajem utilizada. Como ficheiros de entrada foi utilizado ficheiros de texto e em formato binário. O formato de texto tornou-se mais simples de visualizar e projetar a lógica de ordenação, por outro lado o formato binário é mais universal e essencial para aprendizagem e aplicabilidade no mundo real. Os algoritmos de ordenação utilizados foram: Bubble Sort que consiste em percorrer o vetor comparando pares vizinhos e trocando-os se estiverem fora de ordem; O Insertion Sort coloca diretamente cada valor na sua posição relativa correta. ou seja, verifica do grupo de números já ordenados, (inicialmente um) e coloca o novo valor entre os números corretos; por fim, o Quick Sort escolhe um valor para pivô e divide os valores em maiores e menores que o pivô, repetindo até ficar ordenado.

2. ALGORITMOS DE ORDENAÇÂO ESTUDADOS
Foram implementados três algoritmos Bubble Sort, Insertion Sort e Quick Sort. O programa escolhe o algoritmo por nome, através de uma classe controladora, o que permite compará-los sobre os mesmos dados. O motivo de escolha dos algoritmos referidos foi por familiaridade, otimização e interesse e curiosidade para estudo.

O Bubble Sort é o algoritmo de ordenação mais conhecido no mundo informático pela sua simplicidade e preformam-se media, mas suficiente para a maioria das utilizações com eficiência geral de O(n^2). Como referido anteriormente, o seu funcionamento baseia-se na comparação entre o valor do comparando ao seu comparador. No caso de o primeiro ser maior que o seguindo há a troca de posições. Este processo repete-se até todas as comparações serem verdadeiras.

O Insertion Sort trabalha a volta de dividir o vetor em dois, sendo uma parte ordenada e a outra por ordenar. Como primeiro passo é assumido um valor para o grupo ordenado, (visto que apenas um valor está sempre ordenado), de seguida é escolhido valores, um a um, do grupo por ordenar e é colocado a posição relativa correta. Ou seja, no primeiro caso é apenas verificar se o valor é maior ou menor que o valor ordenado. Numa segunda iteração, e nos consequentes, é escolhido se é maior menor ou intermedio aos valores do grupo ordenado. Tem uma taxa de eficiência média de O(n^2) mas no melhor caso com uma taxa de O(n).

Por fim, o Quick Sort tem como base escolher um valor como pivô e distribui os valores restantes como maiores ou menores que o pivô, dividindo assim os valores em dois, (maiores ou menores). Os passos seguintes é a repetição de este conceito, neste caso, um no lado menor que o pivô original e ou no lado maior, repetindo até o vetor ser ordenado. Este algoritmo tem uma eficiência de O(n logn) com um pior caso de O(n^2)

3. IMPLEMENTAÇÃO DO TRABALHO
O programa implementado lê um ficheiro de inteiros, ordena-os em memória e grava o resultado num novo ficheiro em outputs/, com o mesmo formato da entrada.

Para correr o programa é utilizado a linha de comandos e é corrido os comandos:

- javac algoritmos\Main.java
    - Este comando serve para compilar o ficheiro e a classe main e todos os ficheiros utilizados
- Java algoritmos.Main (nome ficheiro a ordenar) (algoritmo de ordenação, escolha entre os referidos antreormente)

Estes comandos têm de ser corridos estando dentro da pasta “TP1_AED_a85234”.

Fluxo de execução:

Após correr os comandos referidos o programa segue o seguinte fluxo:

1. Decide o formato pela extensão do ficheiro: *”.txt*” é texto, qualquer outra é binário.
2. Determina o número de inteiros “n”.
3. Lê os n inteiros para um int[].
4. Ordena o vetor com o algoritmo referido no comando.
5. Constrói o nome do ficheiro de saída: (outputs/<nome>_ordenado<extensão>) e grava o vetor ordenado no mesmo formato.
6. Acrescenta a log.txt uma linha com data, hora, algoritmo, número de elementos e duração da ordenação

Estrutura e Justificação:

Cada responsabilidade está numa classe e ficheiro próprio para organização, clareza de código e facilidade de alterar e manipular cada componente.

```
TP1_AED_a85234/
├── algoritmos/
│   ├── Main.java
│   ├── Log.java
│   ├── Conversor.java
│   ├── R_W/
│   │   ├── Leitura.java
│   │   └── Escrita.java
│   └── alg_ordenacao/
│       ├── ControladorAlgoritmos.java
│       ├── BubbleSort.java
│       ├── InsertionSort.java
│       └── QuickSort.java
├── inputs/
├── outputs/
├── log.txt
└── README.md
```

O Main tem como função principal chamar cada modulo do programa. Representando o fluxo de forma bastante direta. Como primeiro passo é inicializado um timer para no fim ser possível verificar o tempo de execução do programa, ou seja, o tempo de execução dos algoritmos.

A primeira classe a ser chamada, após o main, é a Leitura. Esta class tem duas funções estando duplicadas para números interios e binários: Ler um ficheiro e informar quantos valores estão no ficheiro .txt ou binário. A função de verificar o numero de inteiros a ordenar consiste em, para txt, verificar quantas linhas estão preenchidas e devolver esse valor. Para binário é recolhido o numero de bits e dividido por 4. É dividido por 4 visto que cada inreiro é formado po 4 bytes logo o total de bites equivale ao total dividido por 4.

Para a leitura em si no caso de .txt é lido através de um bufferedReader e FileReader o ficheiro correndo “n” linhas equivalente ao numero de interios a ordenar e são convertidos para Int e guardados num vetor. No caso de binário é utilizado o DataInputStream e FileInputStream em que lê 4 bytes transformando a string de binários em Int e são guardados num vetor. No fim para ambos os casos a class devolve o vetor.

Voltando ao Main, é utilizado o algoritmo escolhido para ordenar o vetor. Para isso é chamado a class associada ao algoritmo. Cada algoritmo vai refletir a sua funcionalidade como referido anteriormente.

De seguida é preparado o ficheiro de output. Neste caso é dividido o nome original do ficheiro no seu nome e extensão e é adicionado o caminho para a pasta outputs com um complemento “_ordenado” e a extensão (binário ou txt).

Por fim é terminado a contagem de tempo e é enviado o algoritmo escolhido, a duração do programa e o numero de argumentos para a class Log função escrever_log em que é adicionado uma linha nova para o ficheiro de texto log.txt contendo esses mesmos dados.

4. ESTRUTURA DO FICHEIRO A ORDENAÇÃO

O programa está preparado para ordenar ficheiros de texto e binários. O ficheiro de texto tem um valor em cada linha tornando fácil a sua leitura. O ficheiro binário tem todos os valores numa string de binário continua. Facilitando a leitura visto que há a possibilidade de dividir o total de valores binários pelo tamanho alocado para números. Neste caso 4 bytes ser descrito o modo de utilização/funcionamento do programa.

5. ESTUDO COMPARATIVO DE DESEMPENHO

O tempo foi medido com System.nanoTime() ao longo de toda a execução do programa (leitura do ficheiro, ordenação e escrita do resultado) e registado em log.txt. Cada algoritmo foi executado 10 vezes sobre o mesmo ficheiro de 5.000 inteiros, e usou-se a média das 10 execuções.

Análise (n = 5.000). A ordem dos algoritmos é clara e estável: em nenhuma das 10 execuções os intervalos se sobrepõem (Bubble Sort entre 55,3 e 57,9 ms, Insertion Sort entre 29,7 e 32,7 ms, Quick Sort entre 23,1 e 25,9 ms). O Quick Sort foi o mais rápido, 2,3 vezes mais que o Bubble Sort e 1,3 vezes mais que o Insertion Sort, e o Insertion Sort foi 1,8 vezes mais rápido que o Bubble Sort.

Isto concorda com o esperado teórico. As diferenças medidas são, no entanto, menores do que as teóricas, porque o tempo registado inclui a leitura e a escrita dos ficheiros,

6. CONCLUSÃO

Até à resolução deste trabalho havia questões as quais não estava habituado a não ter uma resposta direta. Por exemplo, ordenar um vetor, ou ordenação no geral, torna-se banal até à necessidade de traduzir esse conceito para

máquina de forma à mesma fazer o pretendido. Neste caso tem de ser especificado, não só a estrutura dos dados e o ficheiro que os contém, mas como também a tradução dos mesmos de forma a poderem ser manipulados, (neste caso com um foco mais virado para os ficheiros binários).

Foi notável as várias formas diferentes de ordenar valores e as suas diferenças de lógica, criatividade e eficiência. Com isto, foi verificado a seguinte performance ao ordenar 5000 valores:

- Bubblesort: 56,26 ms
- Insertion sort: 31,29 ms
- Quicksort: 24,02 ms

Verificando uma maior rapidez na utilização do Quick Sort.

7. AGRADECIMENTOS

Gostava de aproveitar a agradecer ao professor Roberto Lam pela prontidão e disponibilidade para com os alunos na resolução de problemas.

8. REFERÊNCIAS

Lam, R. C. (s.d.). Aula PL4 e TI.

Lam, R. C. (s.d.). POO_JavaStreams.

Lam, R. C. (s.d.). Programação Orientada a objetos.

Lam, R. C. (s.d.). Programação Orientada a Objetos (2).